Aula prática

Triggers no Oracle

Aprenda a criar, testar, inspecionar e controlar triggers DML usando o Oracle FreeSQL.

Público	Duração	Ambiente
Iniciantes em SQL e PL/SQL	75 a 90 minutos	Oracle FreeSQL, no navegador

Objetivo de aprendizagem: ao final, a turma conseguirá reconhecer quando uma trigger é disparada, entender seus dados de contexto e criar exemplos seguros de validação e auditoria.

1. O que é uma trigger?

Uma trigger é um bloco PL/SQL armazenado no banco e executado automaticamente quando ocorre um evento definido. Ela não é chamada com EXEC: quem a dispara é um comando como INSERT, UPDATE ou DELETE.

Fluxo de funcionamento



Vocabulário essencial

Elemento	Significado
BEFORE / AFTER	A ação ocorre antes ou depois do comando.
FOR EACH ROW	A trigger executa uma vez para cada linha afetada.
:NEW	Valor novo da linha.
:OLD	Valor anterior da linha.

Regra prática: priorize restrições nativas como NOT NULL, CHECK, UNIQUE e chaves estrangeiras para regras simples. Triggers são especialmente úteis para auditoria e ações automáticas justificadas.

2. Preparação no FreeSQL

No Oracle FreeSQL, entre em seu schema pessoal e execute o bloco abaixo. Ele cria a tabela principal, a tabela de auditoria e uma sequência para gerar identificadores.

```
CREATE TABLE produto (
id_produto NUMBER PRIMARY KEY,
nome VARCHAR2(100) NOT NULL,
preco NUMBER(10,2) NOT NULL
);

CREATE TABLE produto_auditoria (
id_auditoria NUMBER PRIMARY KEY,
id_produto NUMBER NOT NULL,
preco_anterior NUMBER(10,2),
preco_novo NUMBER(10,2),
alterado_em TIMESTAMP NOT NULL
);

CREATE SEQUENCE seq_produto_auditoria START WITH 1;
```

Dica para a turma: execute cada comando CREATE isoladamente se o editor não aceitar todo o bloco de uma vez.

3. Demonstração 1 - BEFORE INSERT

Vamos validar o preço antes de a linha ser gravada. Se o preço for negativo, a trigger gera um erro e o `INSERT` falha.

```
CREATE OR REPLACE TRIGGER trg_produto_preco_bi
BEFORE INSERT ON produto
FOR EACH ROW
BEGIN
IF :NEW.preco < 0 THEN
RAISE_APPLICATION_ERROR(-20001, 'O preço não pode ser negativo.');
END IF;
END;
/
```

```
INSERT INTO produto (id_produto, nome, preco)
VALUES (1, 'Teclado', 120);

-- Este comando deve falhar com ORA-20001:
INSERT INTO produto (id_produto, nome, preco)
VALUES (2, 'Produto inválido', -5);
```

Pergunta para discussão: por que a trigger é `BEFORE`, e não `AFTER`?

4. Demonstração 2 - Auditoria de preço

Em uma trigger de linha, `:OLD` representa o dado anterior e `:NEW` o novo. Aqui, guardamos as mudanças de preço em uma tabela de auditoria.

```
CREATE OR REPLACE TRIGGER trg_produto_aud_au
AFTER UPDATE OF preco ON produto
FOR EACH ROW
WHEN (OLD.preco <> NEW.preco)
BEGIN
INSERT INTO produto_auditoria (
id_auditoria, id_produto, preco_anterior, preco_novo, alterado_em
) VALUES (
seq_produto_auditoria.NEXTVAL,
:OLD.id_produto, :OLD.preco, :NEW.preco, SYSTIMESTAMP
);
END;
/
```

```
UPDATE produto SET preco = 135 WHERE id_produto = 1;
SELECT * FROM produto_auditoria;
```

O evento só é relevante para alterações que mencionam `preco`; o `WHEN` impede uma auditoria quando o valor permanece igual.

5. Demonstração 3 - Bloqueio de DELETE

Uma trigger também pode bloquear uma operação. Este é apenas um exemplo didático: em produção, considere cuidadosamente se uma exclusão deve ser impedida, arquivada ou controlada por permissões.

```
CREATE OR REPLACE TRIGGER trg_produto_bloqueia_bd
BEFORE DELETE ON produto
FOR EACH ROW
BEGIN
RAISE_APPLICATION_ERROR(-20002, 'Produtos não podem ser excluídos.');
END;
/

-- Deve falhar com ORA-20002:
DELETE FROM produto WHERE id_produto = 1;
```

```
ALTER TRIGGER trg_produto_bloqueia_bd DISABLE;
ALTER TRIGGER trg_produto_bloqueia_bd ENABLE;
```

6. Inspeção e manutenção

Triggers são criadas habilitadas por padrão. Para conhecer as triggers do seu schema e administrar uma delas:

```
SELECT trigger_name, triggering_event, trigger_type, status
FROM user_triggers
ORDER BY trigger_name;

DROP TRIGGER trg_produto_bloqueia_bd;
```

Pontos de atenção

- Em `WHEN`, use `OLD.coluna` e `NEW.coluna` sem dois-pontos. No corpo PL/SQL, use `:OLD` e `:NEW`.
- `:NEW` não existe em `DELETE`; `:OLD` não existe em `INSERT`.
- Evite consultar ou alterar a própria tabela em uma trigger de linha: isso pode provocar `ORA-04091` (tabela mutante).
- Uma trigger que lança erro faz o comando que a disparou falhar.

7. Exercícios

1. Crie uma trigger que impeça `UPDATE` de um produto para preço menor que zero.
2. Acrescente a coluna `usuario` à auditoria e grave `USER` na trigger.
3. Crie uma tabela `pedido` e uma trigger que impeça pedidos com total zero ou negativo.
4. Desafio: faça uma auditoria de exclusões com `AFTER DELETE` e `:OLD`.

Checklist de encerramento

Para cada trigger, o aluno deve saber responder: qual evento a dispara, em que momento, para qual linha e qual ação ela executa?

Referências: Oracle FreeSQL - https://www.oracle.com/database/technologies/oracle-free-sql/ | Oracle Database Documentation - About Triggers e CREATE TRIGGER.